LAPORAN PRAKTIKUM
Komunikasi Inter-service dengan gRPC (Polyglot)

Nama	Gibran Surya Gunanda
NIM	2024903430084
Mata Kuliah	Paradigma Sistem untuk IT
Topik	Hands-on Arsitektur Monolith vs Microservices
Bahasa / Framework	Python 3 / Flask
Lingkungan	Windows, PowerShell, Virtual Environment (venv)
reposity	https://github.com/Gunandaa/grpc-polyglot-lab

Program Studi Teknologi Rekayasa Komputer Jaringan
Jurusan Teknologi Informasi dan Komputer
Politeknik Negeri Lhokseumawe
2026

LEMBAR PENGESAHAN

No. Praktikum	:	02
Judul Praktikum	:	Komunikasi Inter-service dengan gRPC (Polyglot)
Tanggal Praktikum	:	12-10-2026
Tanggal Penyerahan	:	19-10-2026
Nama Praktikan	:	Gibran Surya Gunanda
NIM/Kelas Praktikan	:	2024903430084 / TRKJ 3C
Nilai Praktikum	:	.....
Dosen Pengampu	:	Muhammad Reza Zulma, S.ST., M.Sc

Mengetahui,
Dosen Pengampu

Muhammad Reza Zulma, S.ST., M.Sc

NIP. 199205012022031005

DAFTAR ISI

BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan Praktikum 4
BAB II DASAR TEORI 5
BAB III LANGKAH KERJA 6
3.1 Alat dan Bahan 6
3.2 Persiapan Lingkungan 6
3.3 Setup Workspace dan Git 7
3.4 Menulis Kontrak API (user.proto) 8
3.5 Implementasi gRPC Server (Go) 9
3.6 Implementasi gRPC Client (Python) 11
3.7 Implementasi gRPC Client (Node.js) 13
3.8 Pengujian 14
3.9 Push ke GitHub 15
3.10 Tugas Evaluasi Mandiri 15
BAB IV HASIL DAN PEMBAHASAN 20
4.1 Hasil Pengujian dan Pembahasan 20
4.2 Kendala dan Solusi 20
BAB V PENUTUP 22
5.1 Kesimpulan 22
5.2 Saran 22
LAMPIRAN 23

BAB I PENDAHULUAN

1.1 Latar Belakang

Layanan mikro modern sering ditulis dalam bahasa pemrograman yang berbeda-beda, sehingga dibutuhkan cara komunikasi antar-layanan yang cepat dan konsisten. gRPC menjawab kebutuhan ini dengan memakai Protocol Buffers sebagai kontrak tunggal yang dapat dikompilasi ke berbagai bahasa. Praktikum ini menerapkan pendekatan tersebut dengan membangun satu server Go yang dipanggil oleh klien Python dan Node.js.

1.2 Tujuan Praktikum

1. Memahami arsitektur *Schema-First Development* menggunakan Protocol Buffers.
2. Mampu menginstalasi komponen compiler gRPC (protoc).
3. Berhasil membuat gRPC Server menggunakan bahasa Go.
4. Berhasil membuat gRPC Client menggunakan Python dan Node.js untuk berkomunikasi dengan server Go yang sama.

BAB II DASAR TEORI

gRPC adalah kerangka kerja *Remote Procedure Call* (RPC) berkinerja tinggi yang dikembangkan oleh Google. Kerangka kerja ini memungkinkan layanan mikro (*microservices*) saling berkomunikasi seolah-olah memanggil fungsi secara lokal, meskipun berjalan pada mesin atau bahasa pemrograman yang berbeda.

Konsep utama gRPC yang digunakan pada praktikum ini adalah sebagai berikut.

- **Protocol Buffers (Protobuf).** Berbeda dengan REST yang memakai JSON berbasis teks, gRPC menggunakan format data biner yang didefinisikan melalui berkas berekstensi `.proto`. Berkas ini berfungsi sebagai kontrak layanan (*service contract*) yang mendefinisikan struktur data dan metode yang tersedia.
- **HTTP/2.** gRPC memanfaatkan HTTP/2 yang mendukung *multiplexing*, komunikasi dua arah secara *real-time*, serta kompresi data sehingga transfer informasi lebih cepat dan ringan dibanding HTTP/1.1.
- **Code Generation.** Dari satu berkas kontrak `.proto`, compiler `protoc` dapat menghasilkan kode klien dan server dalam berbagai bahasa pemrograman sehingga konsistensi tipe data terjaga dan integrasi lintas platform lebih cepat.
- **Pola komunikasi fleksibel.** gRPC mendukung pola *unary*, *server streaming*, *client streaming*, dan *bidirectional streaming*. Praktikum ini menggunakan pola *unary* (satu permintaan, satu respons).

BAB III LANGKAH KERJA

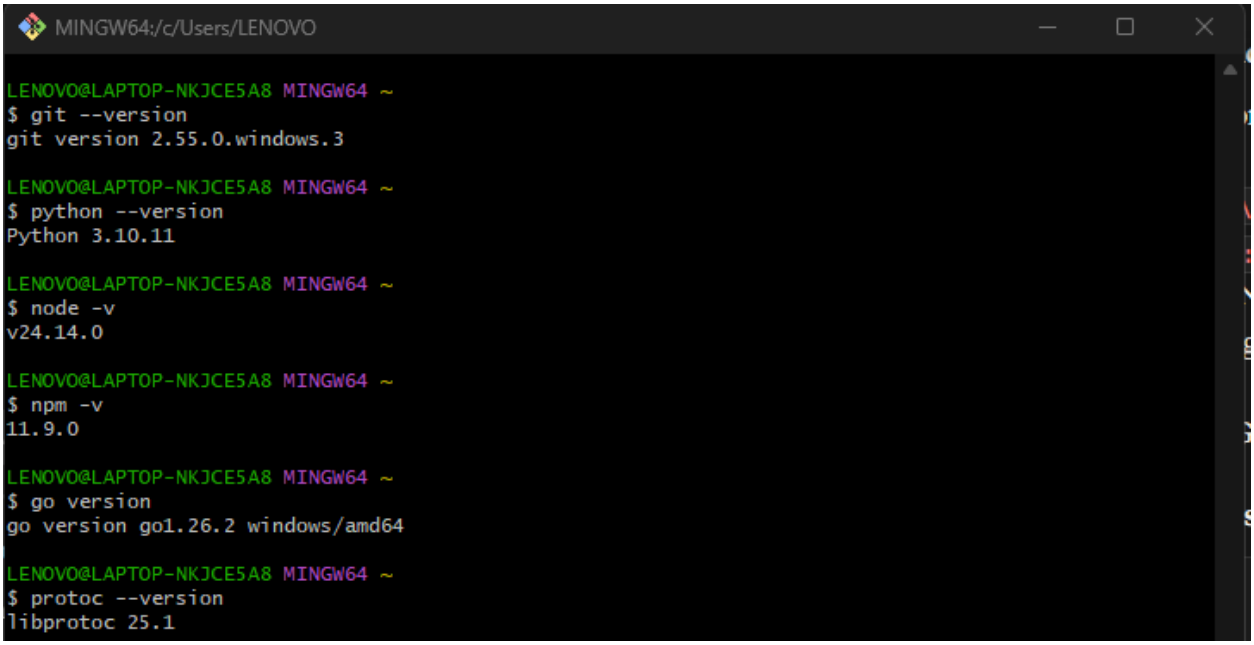
3.1 Alat dan Bahan

No	Perangkat	Keterangan
1	Sistem operasi	Windows
2	Terminal	Git Bash
3	Git	Version control
4	Python	Versi 3.10 atau lebih baru, beserta `grpcio` dan `grpcio-tools`
5	Node.js	Versi LTS, beserta `@grpc/grpc-js` dan `@grpc/proto-loader`
6	Go (Golang)	Beserta `google.golang.org/grpc` dan `google.golang.org/protobuf`
7	Protocol Buffers Compiler	`protoc` untuk Windows (win64)
8	Plugin Go	`protoc-gen-go` dan `protoc-gen-go-grpc`
9	Editor	VS Code atau editor teks lainnya

3.2 Persiapan Lingkungan

Semua perangkat lunak yang dibutuhkan dipasang terlebih dahulu. Khusus `protoc`, berkas `protoc-xx.x-win64.zip` diunduh dari halaman rilis protobuf di GitHub, diekstrak ke `C:\protoc`, lalu direktori `C:\protoc\bin` dan `%USERPROFILE%\go\bin` didaftarkan ke PATH. Setelah itu seluruh instalasi diverifikasi menggunakan perintah berikut.

```
git --version
python --version
node -v
npm -v
go version
protoc --version
```



Gambar 1. Verifikasi instalasi seluruh perangkat.

3.3 Setup Workspace dan Git

Folder proyek `grpc-polyglot-lab` dibuat dan diinisialisasi sebagai repository Git. Berkas `.gitignore` dibuat agar dependensi dan berkas hasil kompilasi lokal tidak ikut terunggah, kemudian dilakukan commit pertama.

```
mkdir grpc-polyglot-lab
cd grpc-polyglot-lab
git init
echo "node_modules/" >> .gitignore
echo "__pycache__/" >> .gitignore
echo ".venv/" >> .gitignore
echo "*.exe" >> .gitignore
git add .gitignore
git commit -m "Initial commit: Add gitignore rules"
```

```
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ echo "node_modules/" >> .gitignore

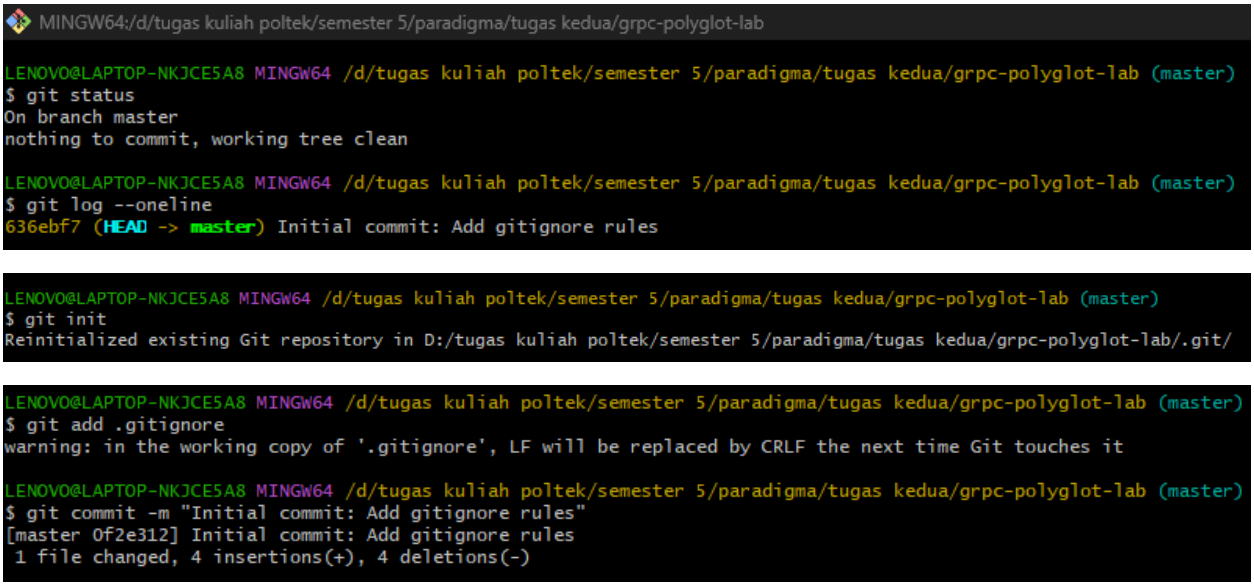
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ echo "__pycache__/" >> .gitignore

LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ echo ".venv/" >> .gitignore

LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ echo "*.exe" >> .gitignore

LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ git add .gitignore
warning: in the working copy of 'grpc-polyglot-lab/.gitignore', LF will be replaced by CRLF the next time Git touches it

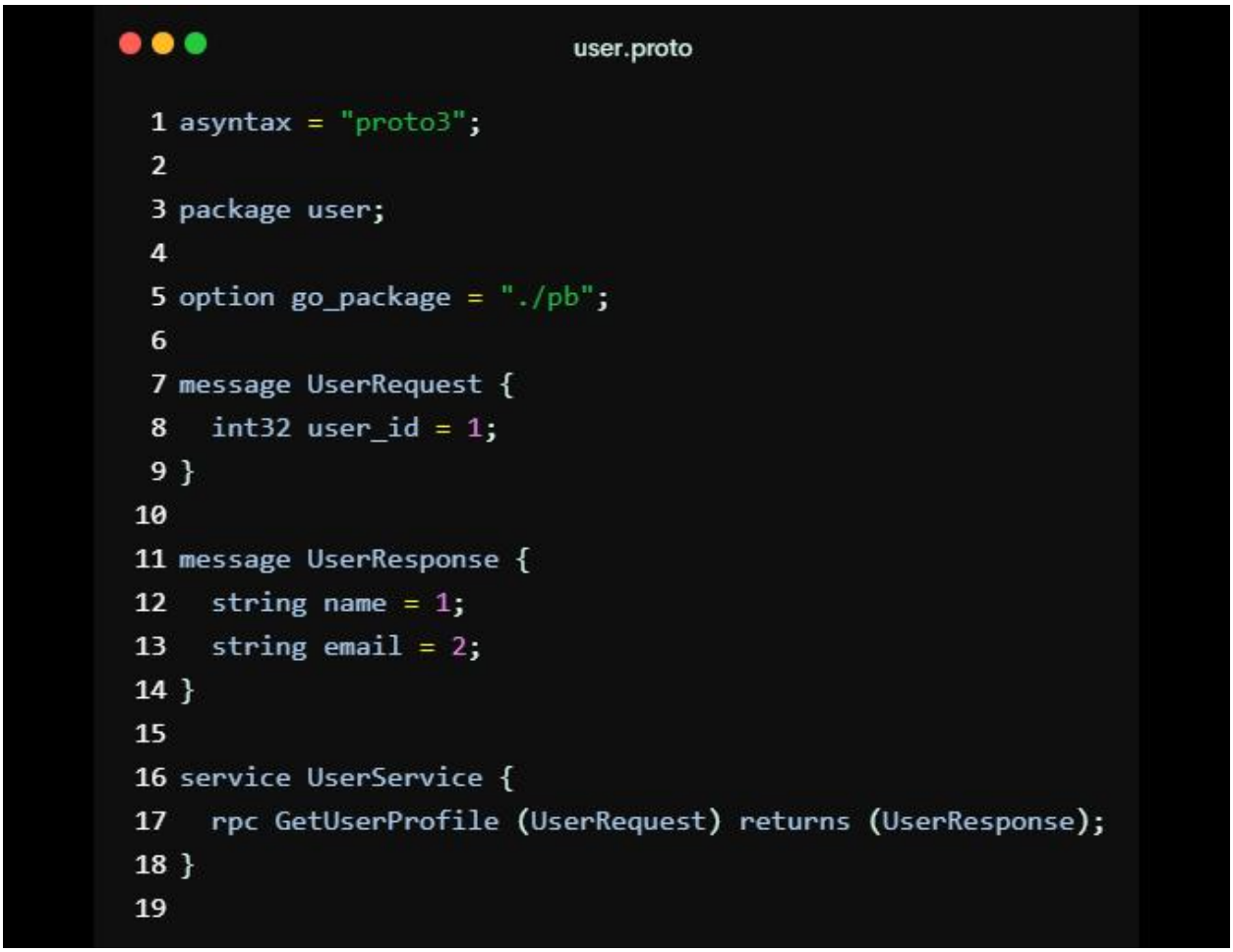
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (master)
$ git commit -m "Initial commit: Add gitignore rules"
[master (root-commit) 47ad5ab] Initial commit: Add gitignore rules
 1 file changed, 4 insertions(+)
 create mode 100644 grpc-polyglot-lab/.gitignore
```



Gambar 2. Inisialisasi repository dan commit pertama.

3.4 Menulis Kontrak API (`user.proto`)

Berkas `user.proto` dibuat di folder utama proyek dan berperan sebagai *single source of truth* untuk seluruh layanan. Kontrak ini mendefinisikan pesan `UserRequest`, pesan `UserResponse`, dan layanan `UserService` dengan metode `GetUserProfile`.



Gambar 3. Kontrak API pada berkas user.proto.

3.5 Implementasi gRPC Server (Go)

Folder `go-server` dibuat, modul Go diinisialisasi, dan dependensi gRPC serta plugin `protoc` dipasang. Berkas `user.proto` kemudian dikompilasi menjadi kode Go.

```
mkdir go-server
cd go-server
go mod init go-server
go get google.golang.org/grpc
go get google.golang.org/protobuf
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
protoc --proto_path=../ --go_out=. --go-grpc_out=. ../user.proto
```

Perintah di atas menghasilkan folder `pb` yang berisi `user.pb.go` dan `user_grpc.pb.go`. Selanjutnya dibuat berkas `main.go` yang menjalankan server pada port 50051. Server mengembalikan data "Budi Santoso" untuk `user_id` bernilai 1, dan pesan "User Tidak Ditemukan" untuk nilai lainnya. Data tersebut ditulis langsung (*hardcoded*) sebagai data tiruan.

```
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (
master)
$ protoc --go_out=go-server --go-grpc_out=go-server user.proto

LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab (
master)
$ ls go-server/pb
user.pb.go  user_grpc.pb.go
```

Gambar 4. Hasil kompilasi proto untuk Go.

user.proto

```go
package main

import (
    "context"
    "fmt"
    "log"
    "net"

    "go-server/pb"

    "google.golang.org/grpc"
)

type server struct {
    pb.UnimplementedUserServiceServer
}

func (s *server) GetUserProfile(ctx context.Context, req *pb.UserRequest) (*pb.UserResponse, error) {
    log.Printf("Menerima request untuk User ID: %d\n", req.GetUserId())
    if req.GetUserId() == 1 {
        return &pb.UserResponse{
            Name:  "Budi Santoso",
            Email: "budi@universitas.ac.id",
        }, nil
    }
    return &pb.UserResponse{
        Name:  "User Tidak Ditemukan",
        Email: "-",
    }, nil
}

func main() {
    lis, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatalf("Gagal listening port: %v", err)
    }
    s := grpc.NewServer()
    pb.RegisterUserServiceServer(s, &server{})
    fmt.Println("gRPC Server (Go) berjalan di port :50051...")
    if err := s.Serve(lis); err != nil {
        log.Fatalf("Gagal menjalankan server: %v", err)
    }
}
```[cite: 1]

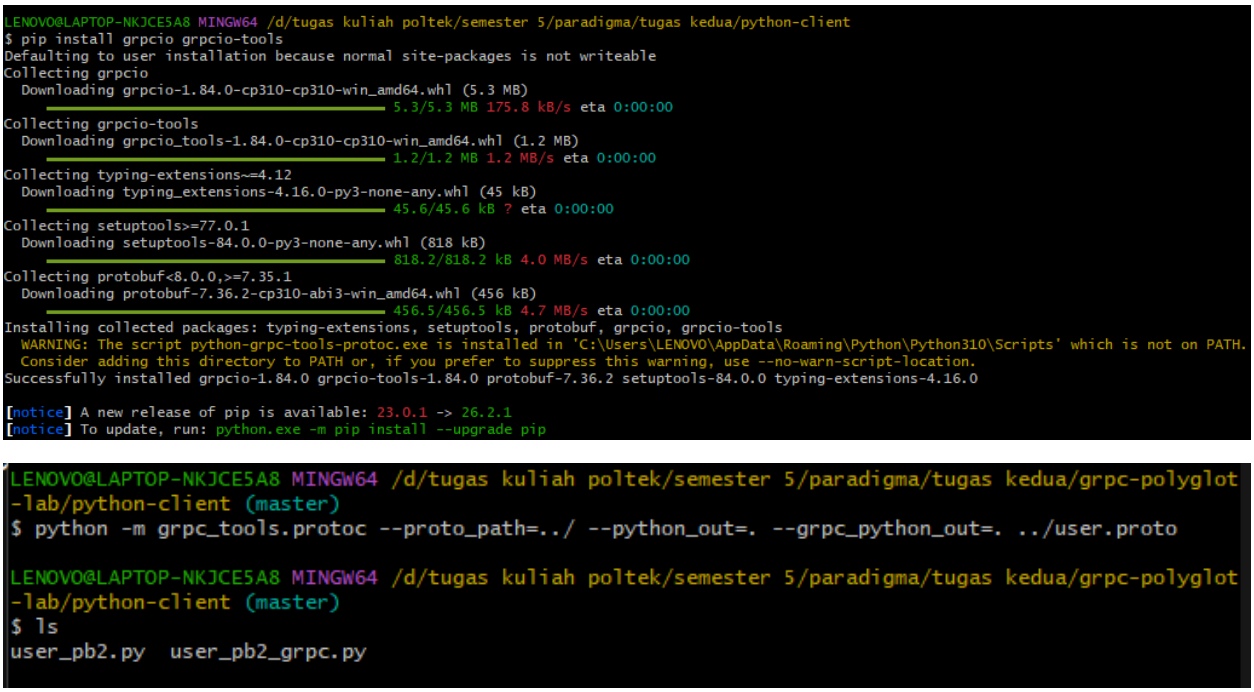
Gambar 5. Kode gRPC Server menggunakan Go.

3.6 Implementasi gRPC Client (Python)

Folder `python-client` dibuat, pustaka dipasang, dan kode Python dihasilkan dari berkas `user.proto` yang sama.

```
cd ..
mkdir python-client
cd python-client
pip install grpcio grpcio-tools
python -m grpc_tools.protoc --proto_path=../ --python_out=. --grpc_python_out=. ../user.proto
```

Proses ini menghasilkan `user_pb2.py` dan `user_pb2_grpc.py`. Berkas `client.py` kemudian dibuat untuk membuka kanal ke `localhost:50051`, memanggil `GetUserProfile`, dan mencetak responsnya.



Gambar 6. Hasil generate kode Python.

client.py

```python
import grpc
import user_pb2
import user_pb2_grpc


def run():
    with grpc.insecure_channel('localhost:50051') as channel:
        stub = user_pb2_grpc.UserServiceStub(channel)
        print("Mengirim request ke Go Server menggunakan Python...")
        response = stub.GetUserProfile(user_pb2.UserRequest(user_id=1))
        print(f"Respon dari Server -> Nama: {response.name}, Email: {response.email}")


if __name__ == '__main__':
    run()
```

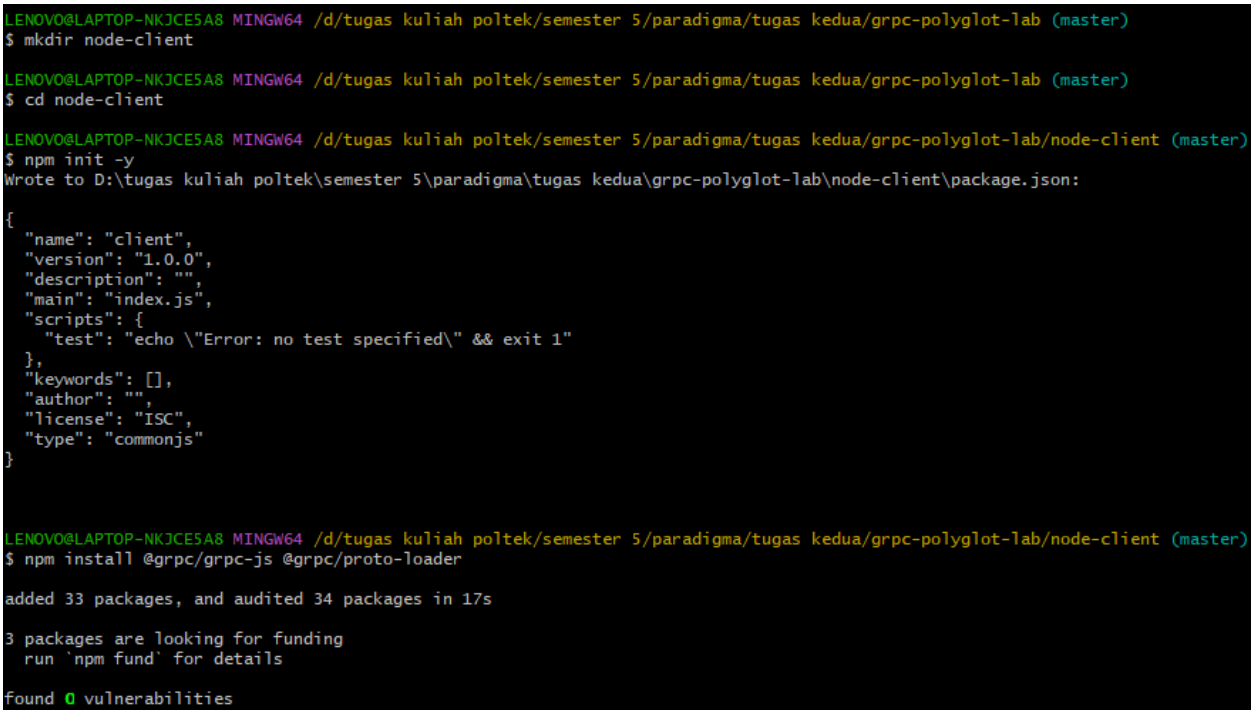
Gambar 7. Kode gRPC Client menggunakan Python.

3.7 Implementasi gRPC Client (Node.js)

Folder `node-client` dibuat dan pustaka gRPC untuk Node.js dipasang.

```
cd ..
mkdir node-client
cd node-client
npm init -y
npm install @grpc/grpc-js @grpc/proto-loader
```

Berbeda dengan Go dan Python, klien Node.js tidak memerlukan proses kompilasi. Berkas `client.js` memanfaatkan `@grpc/proto-loader` untuk membaca `user.proto` secara dinamis saat program dijalankan.



Gambar 8. Instalasi pustaka gRPC untuk Node.js.

client.js

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');
const path = require('path');

const PROTO_PATH = path.join(__dirname, '../user.proto');
const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
    keepCase: true,
    longs: String,
    enums: String,
    defaults: true,
    oneofs: true
});
const userProto = grpc.loadPackageDefinition(packageDefinition).user;

function main() {
    const client = new userProto.UserService('localhost:50051', grpc.credentials.createInsecure());
    console.log("Mengirim request ke Go Server menggunakan Node.js...");
    client.GetUserProfile({ user_id: 1 }, (error, response) => {
        if (!error) {
            console.log(`Respon dari Server -> Nama: ${response.name}, Email: ${response.email}`);
        } else {
            console.error("Terjadi error:", error);
        }
    });
}

main();
```

Gambar 9. Kode gRPC Client menggunakan Node.js.

3.8 Pengujian

Pengujian dilakukan menggunakan tiga terminal terpisah.

Terminal	Perintah	Fungsi
1	cd go-server lalu go run main.go	Menjalankan server Go
2	cd python-client lalu python client.py	Menjalankan client Python
3	cd node-client lalu node client.js	Menjalankan client Node.js

hasil:

node.client

```
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab/node-client (master)
$ node client.js
Mengirim request ke Go Server menggunakan Node.js...
Respon dari Server -> Nama: Budi Santoso, Email: budi@universitas.ac.id
```

python-client

```
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab/python-client (master)
$ python client.py
Mengirim request ke Go Server menggunakan Python...
Respon dari Server -> Nama: Budi Santoso, Email: budi@universitas.ac.id
```

go-server

```
LENOVO@LAPTOP-NKJCE5A8 MINGW64 /d/tugas kuliah poltek/semester 5/paradigma/tugas kedua/grpc-polyglot-lab/go-server (master)
$ go run main.go
gRPC Server (Go) berjalan di port :50051...
2026/10/19 03:51:26 Menerima request untuk User ID: 1
2026/10/19 03:52:11 Menerima request untuk User ID: 1
```

Gambar 10. Hasil pengujian komunikasi antar-layanan.

3.9 Push ke GitHub

Repository kosong dibuat di GitHub, lalu seluruh riwayat kode lokal diunggah.

```
git add .
git commit -m "Feat: Add full polyglot implementation of Go server, Python client, and Node client"
git branch -M main
git remote add origin https://github.com/USERNAME/NAMA_REPO.git
git push -u origin main
```

3.10 Tugas Evaluasi Mandiri

3.10.1 Modifikasi Kontrak Data (Field phone)

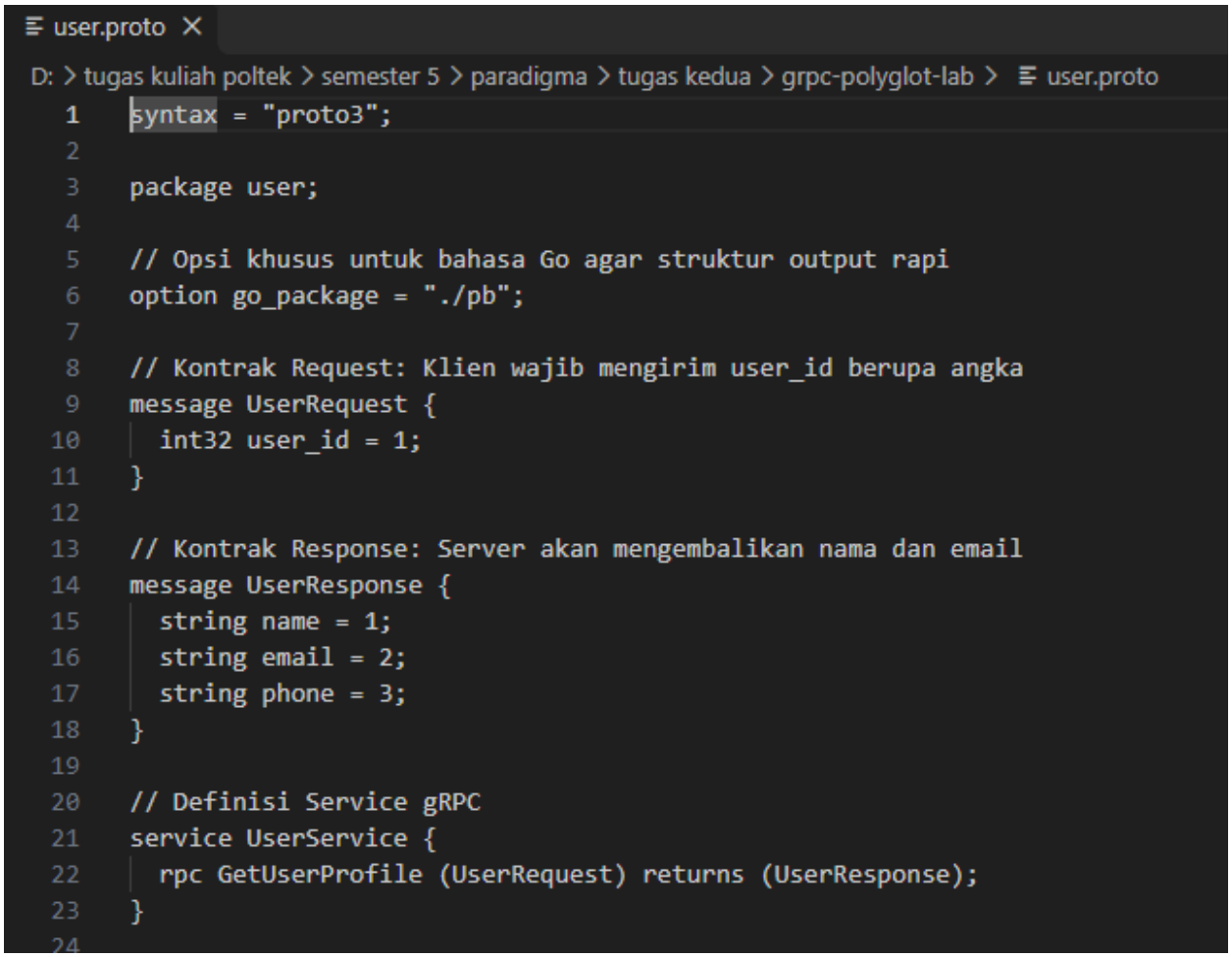
Berkas `user.proto` dimodifikasi dengan menambahkan field `string phone = 3;` pada pesan `UserResponse`.

```
message UserResponse {
  string name = 1;
  string email = 2;
  string phone = 3;
}
```

Setelah itu dilakukan langkah-langkah berikut.

1. Kode Go di-*generate* ulang dengan `protoc`, lalu `main.go` diubah agar mengisi `Phone: "081234567890"` untuk pengguna dengan ID 1 dan `Phone: "-"` untuk pengguna yang tidak ditemukan.
2. Kode Python di-*generate* ulang dengan `grpc_tools.protoc`, lalu `client.py` diubah agar mencetak `response.phone`.

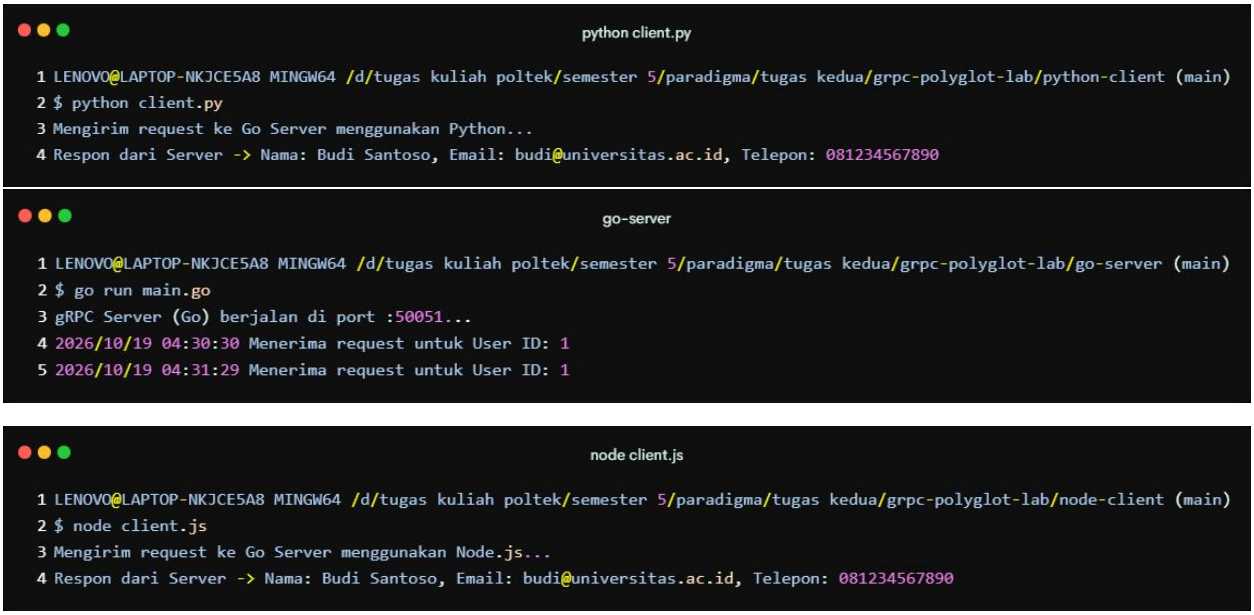
3. Pada Node.js tidak diperlukan proses generate ulang karena skema dibaca saat runtime. Cukup `client.js` yang diubah agar mencetak `response.phone`.
4. Server di-*restart* dan kedua klien dijalankan kembali.



Gambar 11. Kontrak data setelah penambahan field phone.



Gambar 12. Perubahan kode pada server dan kedua klien.



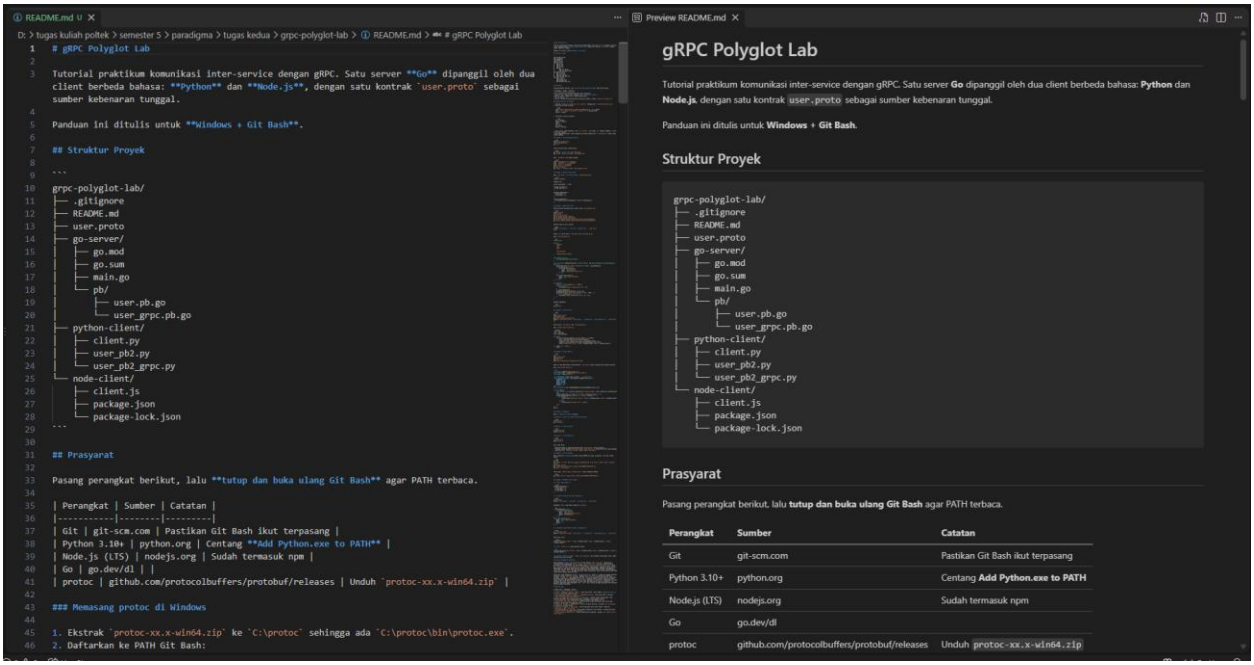
Gambar 13. Hasil pengujian setelah penambahan field phone.

3.10.2 Analisis Pendekatan Kompilasi

Pada pendekatan *static code generation* (Go dan Python), berkas `user.proto` dikompilasi terlebih dahulu menggunakan `protoc` (atau `grpc_tools.protoc` pada Python) sebelum program dijalankan. Hasil kompilasinya berupa berkas kode sumber nyata, yaitu `user.pb.go` dan `user_grpc.pb.go` untuk Go serta `user_pb2.py` dan `user_pb2_grpc.py` untuk Python, yang kemudian diimpor langsung ke dalam program. Keuntungannya, tipe data dapat diperiksa saat pengembangan atau kompilasi, dukungan *autocomplete* lebih baik, dan performa lebih optimal karena kode serialisasi sudah tersedia. Kekurangannya, setiap kali skema `.proto` berubah, pengembang wajib menjalankan ulang proses generate agar kode tetap sinkron dengan kontrak terbaru.

Sebaliknya, pada pendekatan *dynamic runtime generation* (Node.js), tidak ada langkah kompilasi terpisah. Pustaka `@grpc/proto-loader` membaca berkas `user.proto` secara langsung saat aplikasi berjalan (`loadSync`), lalu membangun definisi service dan message di dalam memori. Akibatnya, perubahan skema seperti penambahan field `phone` langsung terbaca setelah klien dijalankan ulang, tanpa proses generate dan tanpa berkas kode hasil kompilasi di repository. Pendekatan ini lebih fleksibel dan cepat untuk prototipe, tetapi tidak ada pemeriksaan tipe saat kompilasi, dan aplikasi bergantung pada keberadaan berkas `.proto` saat runtime sehingga kesalahan baru diketahui ketika program dijalankan.

Analisis ini juga disimpan pada berkas `README.md` di repository.



Gambar 14. Berkas README.md berisi analisis pendekatan kompilasi.

3.10.3 Commit dan Push Kedua

```
git add .
git commit -m "Feat: Add phone field, update clients, and add README analysis"
git push
```

BAB IV HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian dan Pembahasan

Pengujian menunjukkan bahwa satu server gRPC yang ditulis dalam Go dapat dilayani oleh dua klien dengan bahasa pemrograman berbeda, yaitu Python dan Node.js, tanpa perubahan apa pun pada sisi server. Hal ini terjadi karena ketiganya mengacu pada kontrak yang sama, yaitu `user.proto`.

Kriteria keberhasilan pada modul terpenuhi sebagai berikut.

No	Kriteria	Hasil
1	Tidak ada *error crash* (*Connection refused* atau *Cannot find module*) pada ketiga terminal	Terpenuhi
2	Data profil "Budi Santoso" tercetak pada terminal Python dan Node.js, padahal data hanya ditulis pada server Go	Terpenuhi
3	Terminal Go Server menampilkan log setiap ada klien yang memanggil	Terpenuhi

Pada tugas evaluasi, penambahan field `phone = 3` membuktikan sifat *schema-first*: perubahan cukup dimulai dari kontrak, kemudian kode Go dan Python di-*generate* ulang, sementara Node.js langsung menyesuaikan karena membaca skema saat runtime. Nomor field pada Protobuf (`= 1`, `= 2`, `= 3`) bersifat unik dan dipakai pada format biner, sehingga menambah field baru dengan nomor baru tidak merusak field yang sudah ada.

4.2 Kendala dan Solusi

No	Kendala	Penyebab	Solusi
1	`bash: protoc: command not found`	`protoc` belum terpasang atau belum terdaftar di PATH	Mengunduh `protoc-xx.x-win64.zip`, mengekstrak ke `C:\protoc`, lalu menambahkan `C:\protoc\bin` dan `%USERPROFILE%\go\bin` ke PATH (melalui `~/.bashrc` atau Environment Variables Windows)
2	`Could not make proto path relative: ../user.proto`	Folder `python-client` terbuat di luar folder utama sehingga `../user.proto` tidak ditemukan	Memindahkan folder ke dalam `grpc-polyglot-lab` dan menjalankan perintah dari dalam subfolder
3	`The current directory is invalid`	Terminal masih terikat pada lokasi lama folder yang sudah dipindahkan	Menutup Git Bash, membukanya kembali, lalu masuk ke folder yang benar
4	`error: remote origin already exists` dan `Repository not found`	Remote `origin` sudah terpasang dengan URL yang salah	Membuat repository kosong di GitHub, lalu mengganti URL dengan `git remote set-url origin <URL>`

No	Kendala	Penyebab	Solusi
5	`syntax error: unexpected literal` pada `main.go`	Baris kode Python tidak sengaja masuk ke berkas Go	Menimpa `main.go` dengan versi yang benar

BAB V PENUTUP

5.1 Kesimpulan

1. Pendekatan *schema-first* dengan Protocol Buffers menjadikan berkas `user.proto` sebagai kontrak tunggal yang dipakai bersama oleh server dan seluruh klien, sehingga konsistensi tipe data antar-bahasa terjaga.
2. Compiler `protoc` berhasil dipasang pada Windows dan digunakan untuk menghasilkan kode Go dan Python secara otomatis dari satu berkas kontrak.
3. gRPC Server berbasis Go berhasil dibuat dan dijalankan pada port 50051.
4. gRPC Client berbasis Python dan Node.js berhasil berkomunikasi dengan server Go yang sama dan menerima data profil pengguna dengan benar.
5. Pendekatan *static code generation* (Go dan Python) menawarkan pemeriksaan tipe dan performa yang baik namun memerlukan proses generate ulang, sedangkan pendekatan *dynamic runtime generation* (Node.js) lebih fleksibel namun tanpa pemeriksaan tipe saat kompilasi.

5.2 Saran

- Pada lingkungan produksi, gunakan koneksi aman (TLS) sebagai pengganti kanal tanpa enkripsi (`insecure_channel`) yang dipakai selama praktikum.
- Kembalikan kode status gRPC (misalnya `NOT_FOUND`) untuk pengguna yang tidak ditemukan, sehingga klien dapat membedakan data kosong dari kegagalan.

LAMPIRAN

- Tautan repository GitHub: [https://github.com/Gunandaa/grpc-polyglot-lab]
- Struktur akhir proyek:

```
grpc-polyglot-lab/
├── .gitignore
├── README.md
├── user.proto
├── go-server/
│   ├── go.mod
│   ├── go.sum
│   ├── main.go
│   └── pb/
│       ├── user.pb.go
│       └── user_grpc.pb.go
├── python-client/
│   ├── client.py
│   ├── user_pb2.py
│   └── user_pb2_grpc.py
└── node-client/
    ├── client.js
    ├── package.json
    └── package-lock.json
```

DAFTAR PUSTAKA

1. gRPC Authors, *gRPC Documentation* (grpc.io)
2. Google, *Protocol Buffers Documentation* (protobuf.dev)
3. The Go Authors, *Go Documentation* (go.dev)
4. Node.js Foundation, *Node.js Documentation* (nodejs.org)
5. Python Software Foundation, *Python Documentation* (docs.python.org)
6. Modul praktikum *Hands-on: Komunikasi Inter-service dengan gRPC (Polyglot)*